

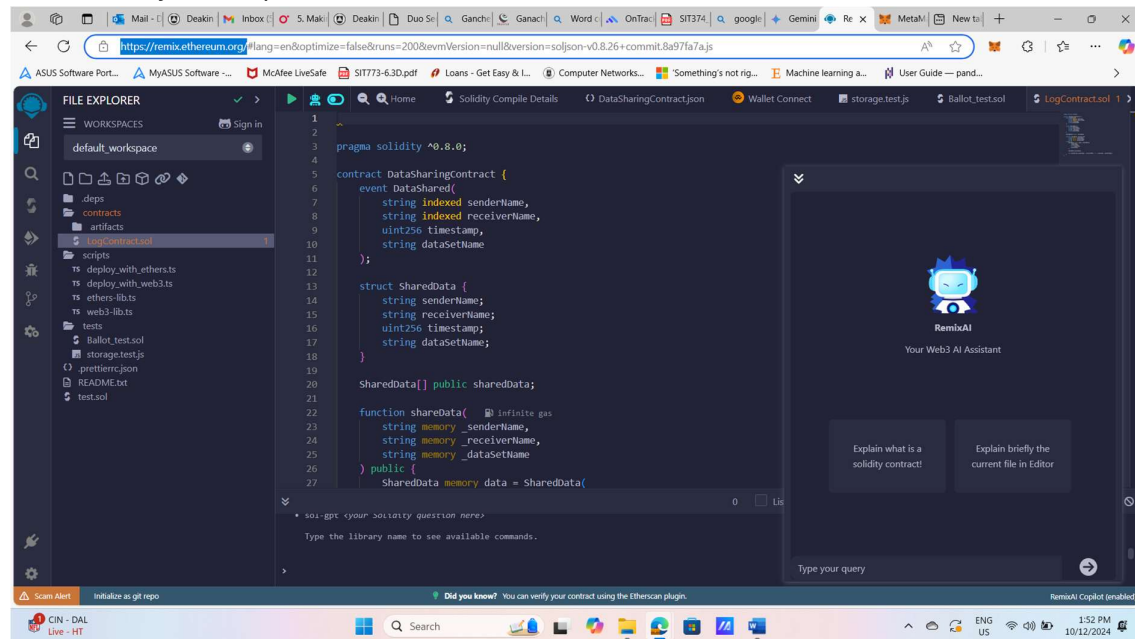
## Documentation for immutable log creation with blockchain.

For testing Brett use case for data sharing with immutable logs we have leveraged blockchain and created a smart contract to test immutable log storing functionality.

## Setting up the development environment :

Remix is an online lightweight code editor for blockchain development.[1]

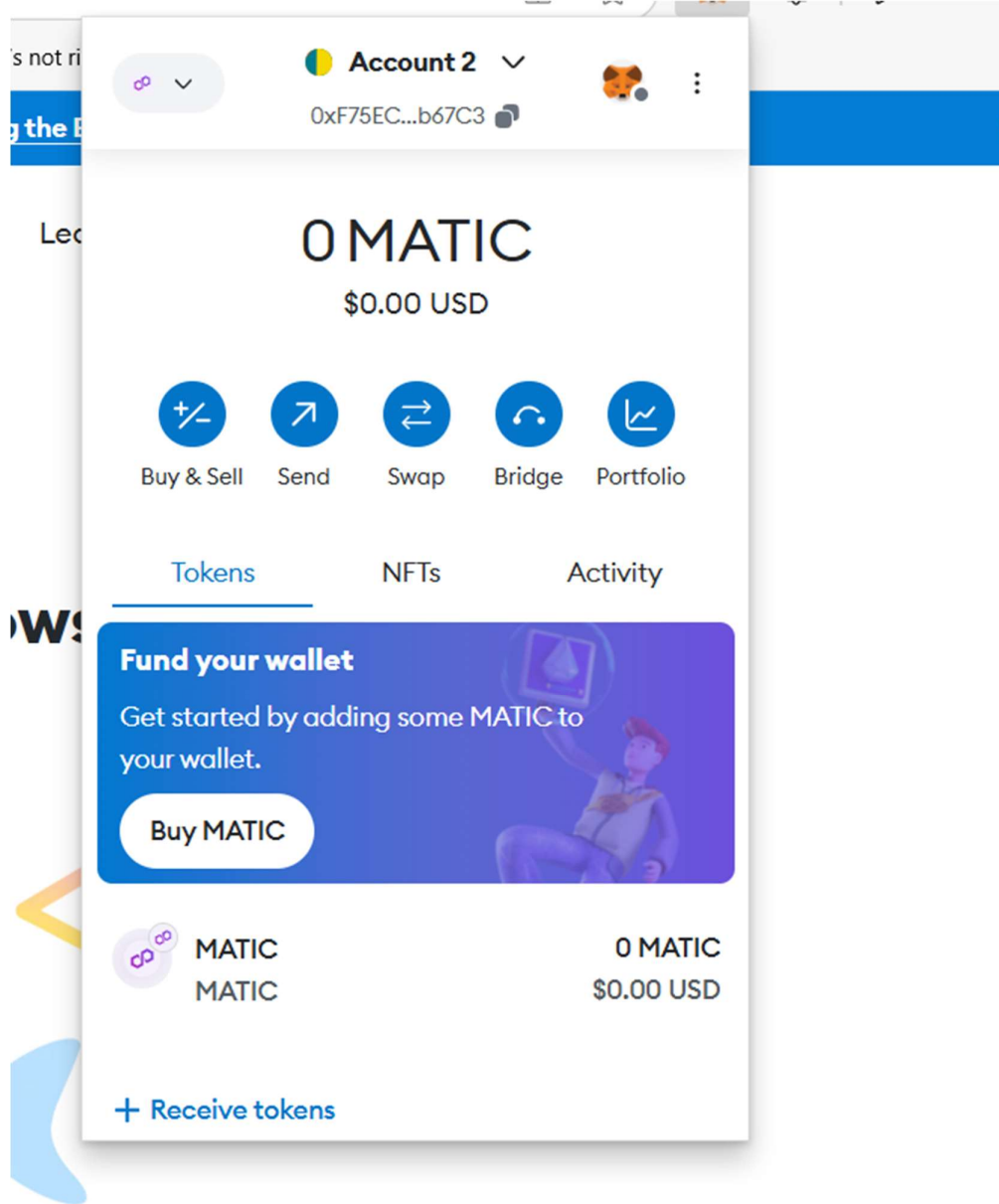
Remix is easy to set up user can create new file in contract section.



## Setting up wallet :

The second step involved in the development of a contract is the wallet. Wallet can be used to store user private keys and interact with blockchain for our project we are using Metamask wallet .[2]

Metamask web browser extension can be downloaded here [3].



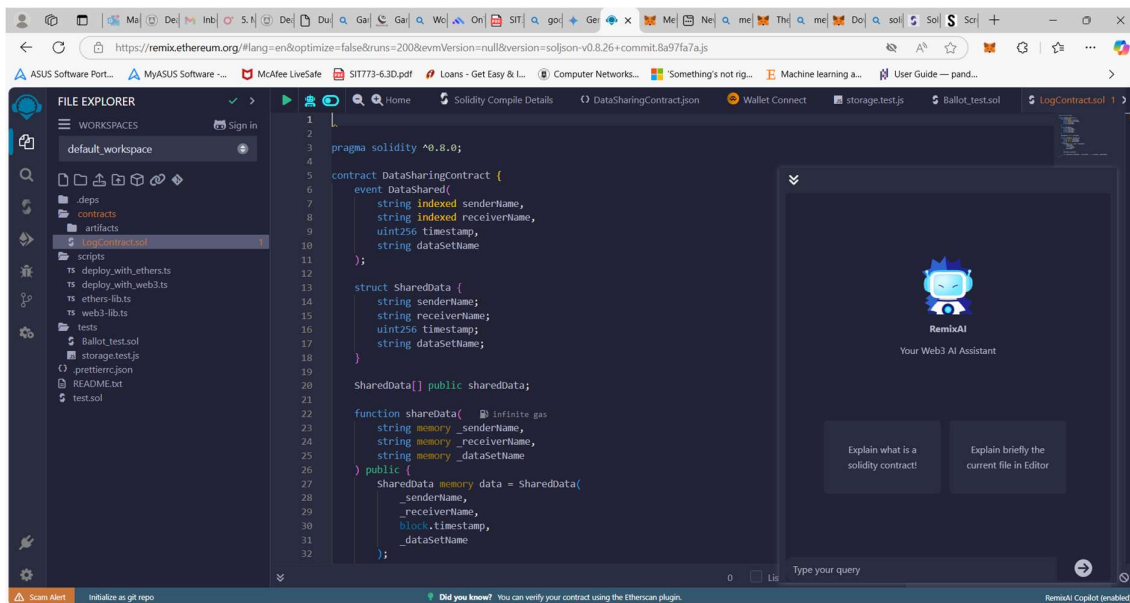
## Understanding Solidity smart contract.

Smart contracts are used to program blockchain and the language is Solidity. Solidity documentation can be found here [4].

Similar to other programming languages solidity has its way of defining data structure, and data types to create programs. Solidity files are stored with .sol extension .

Using solidity we have created LogContract.sol which can be used to store logs in blockchains.

The code below we have to store is given below.



```
1  
2  
3 pragma solidity ^0.8.0;  
4  
5 contract DataSharingContract {  
6     event DataShared(  
7         string indexed senderName,  
8         string indexed receiverName,  
9         uint256 timestamp,  
10        string dataSetName  
11    );  
12  
13    struct SharedData {  
14        string senderName;  
15        string receiverName;  
16        uint256 timestamp;  
17        string dataSetName;  
18    }  
19  
20    SharedData[] public sharedData;  
21  
22    function shareData( @infinite gas  
23        string memory _senderName,  
24        string memory _receiverName,  
25        string memory _dataSetName  
26    ) public {  
27        SharedData memory data = SharedData(  
28            _senderName,  
29            _receiverName,  
30            block.timestamp,  
31            _dataSetName  
32        );  
33    }
```

```
pragma solidity ^0.8.0;
```

```
contract DataSharingContract {  
    event DataShared(  
        string indexed senderName,  
        string indexed receiverName,  
        uint256 timestamp,  
        string dataSetName  
    );
```

```
    struct SharedData {  
        string senderName;  
        string receiverName;  
        uint256 timestamp;  
        string dataSetName;  
    }  
}
```

```
    SharedData[] public sharedData;
```

```

function shareData(
    string memory _senderName,
    string memory _receiverName,
    string memory _dataSetName
) public {
    SharedData memory data = SharedData(
        _senderName,
        _receiverName,
        block.timestamp,
        _dataSetName
    );

    sharedData.push(data);

    emit DataShared(_senderName, _receiverName, block.timestamp, _dataSetName);
}
}

```

### **Deployment**

Deployment is done on Ethereum test net spolia can be found here [Contract Address 0xeb723C91Fe51d69b0AA74Da45be9DA9Eb27B2Fcd | Etherscan.](#)

Since contract is now stored in blockchain its immutable and we have also stored some logs in following transactions

Logs are stored in the following transaction [Sepolia Transaction Hash \(Txhash\) Details | Etherscan.](#) 0x92835a2b91162b9b4045f110cea937ea1675fe8abd7880c3cde2e34fc87dfed6

Etherscan is an explorer to explore blockchain transactions done on a contract.[5]

### **ABI**

Abstract binary interface

When a smart contract is compiled, the compiler generates two outputs:

1. Bytecode: The machine-executable code that is deployed to the blockchain.

2. ABI: A JSON file that describes the contract's interface.

The ABI file contains information about:

- Function Signatures: The names, parameters, and return types of the contract's functions.
- Event Signatures: The names, parameters, and indexed parameters of the contract's events.
- Constructor: The constructor function and its parameters.

```
[
  {
    "anonymous": false,
    "inputs": [
      {
        "indexed": true,
        "internalType": "string",
        "name": "senderName",
        "type": "string"
      },
      {
        "indexed": true,
        "internalType": "string",
        "name": "receiverName",
        "type": "string"
      },
      {
        "indexed": false,
        "internalType": "uint256",
        "name": "timestamp",
        "type": "uint256"
      },
      {
        "indexed": false,
        "internalType": "string",
```

```

        "name": "dataSetName",
        "type": "string"
    }
],
"name": "DataShared",
"type": "event"
},
{
    "inputs": [
        {
            "internalType": "string",
            "name": "_senderName",
            "type": "string"
        },
        {
            "internalType": "string",
            "name": "_receiverName",
            "type": "string"
        },
        {
            "internalType": "string",
            "name": "_dataSetName",
            "type": "string"
        }
    ],
    "name": "shareData",
    "outputs": [],
    "stateMutability": "nonpayable",
    "type": "function"
},
{

```

```
"inputs": [  
  {  
    "internalType": "uint256",  
    "name": "",  
    "type": "uint256"  
  }  
,  
  "name": "sharedData",  
  "outputs": [  
    {  
      "internalType": "string",  
      "name": "senderName",  
      "type": "string"  
    },  
    {  
      "internalType": "string",  
      "name": "receiverName",  
      "type": "string"  
    },  
    {  
      "internalType": "uint256",  
      "name": "timestamp",  
      "type": "uint256"  
    },  
    {  
      "internalType": "string",  
      "name": "dataSetName",  
      "type": "string"  
    }  
  ],  
  "stateMutability": "view",
```

```
        "type": "function"  
    }  
]
```

Abi and deployed contract address can be used to connect smart contract with web application which can be used to connect web application to smart contract

### **References**

- [1].<https://remix.ethereum.org/>
- [2][The Ultimate Crypto Wallet for DeFi, Web3 Apps, and NFTs | MetaMask](#)
- [3] [Download MetaMask: The Premier Blockchain Wallet App and Browser Extension](#)
- [4][Solidity — Solidity 0.8.28 documentation](#)
- [5]<https://sepolia.etherscan.io/>